

BRAC UNIVERSITY
Department of Computer Science and Engineering

Examination: Mid Semester Exam
Duration: 1 Hour

Semester :Fall 2022
Full Marks: 30

CSE 221: Algorithms

Set: A

Answer the following questions.
Figures in the right margin indicate marks.

Name:	ID:	Section:
-------	-----	----------

Total Marks : 10

Jack loves to play with integers. He created a list of n integers where the even indices hold numbers in decreasing order and the odd indices hold numbers in increasing order. For example, this is a list of $n=8$ integers Jack made.

Index	0	1	2	3	4	5	6	7
Number	23	2	19	3	7	11	5	13

[Explanation:

The indices 1, 3, 5, and 7 have numbers 2, 3, 11, and 13 in increasing order.

The indices 0, 2, 4, and 6 have numbers 23, 19, 7 and 5 in decreasing order.]

You sorted the the list in quadratic time [$O(n^2)$]. To your utter surprise, Jack replies, "It could be sorted in linear time".

- 1) **[CO2] Show** the steps how you sorted the list using a suitable algorithm.

Mention the name of the algorithm. —

3

- 2) **[CO2] Describe** how Jack sorted the list in linear time. Show the steps too.— 4

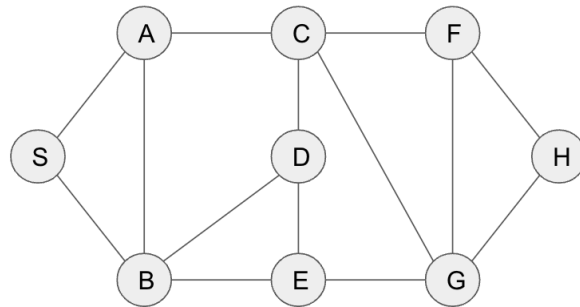
- 3) **[CO3]** Jack wants to add the number 15 to your sorted list from (1). He wants you to come up with an efficient idea about how he is going to find the accurate index position for 15 in the list.

Describe how you are going to assist him in this. **Show** the steps too. —

3

Total Marks: 10[CO3]

Consider this graph:



- Justify** $\sum_{v \in V} \deg(v) = 2m$, where m = number of edges in the graph. 2
- Create** an adjacency matrix for this graph. 2
- Apply** Depth First Search to traverse this graph considering S to be the source vertex. **Demonstrate** the results. 6

Total Marks:10

a.[CO7]Calculate the time complexity of the following function

Finding_Worst_Case(n):

```
int i,j,k,m,multi,a,b,c
for( i = n; i >= 1; i = i / 7 ){
    for( j = 1; j <= n; j = j + 3 ) {
        for( k=1; k<=40 ; k=k+1){
            multi=a*b
        }
        for( m=n ; m>=1 ; m=m-5 ){
            multi=multi*c
        }
    }
}
```

b.[CO7] **Calculate** the time complexity of the following recurrence relation.

[Any method is acceptable as long as steps are shown]

$$T(n) = T(n/2) + T(n/4) + n$$

BRAC UNIVERSITY
Department of Computer Science and Engineering

Examination: Mid Semester Exam
Duration: 1 Hour

Semester :Fall 2022
Full Marks: 30

CSE 221: Algorithms

Set: B

Answer the following questions.
Figures in the right margin indicate marks.

Name:	ID:	Section:
-------	-----	----------

Total Marks : 10

Jack loves to play with integers. He created a list of n integers where the even indices hold numbers in decreasing order and the odd indices hold numbers in increasing order. For example this is a list of n=8 integers Jack made.

Index	0	1	2	3	4	5	6	7
Number	33	7	29	11	17	23	13	31

[Explanation:

The indices 1, 3, 5, and 7 have numbers 7, 11, 23, and 31 in increasing order.

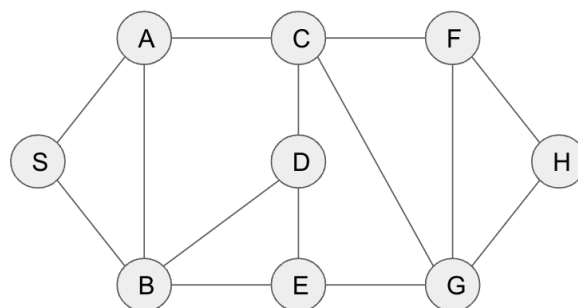
The indices 0, 2, 4, and 6 have numbers 33, 29, 17, and 13 in decreasing order.]

You sorted the the list in quadratic time [$O(n^2)$]. To your utter surprise, Jack replies, “It could be sorted in linear time”.

- 1) **[CO2]** Show the steps how you sorted the list using a suitable algorithm. Mention the name of the algorithm. —
3
- 2) **[CO2]** Describe how Jack sorted the list in linear time. Show the steps too. —
4
- 3) **[CO3]** Jack wants to add the number 12 to your sorted list from (1). He wants you to come up with an efficient idea about how he is going to find the accurate index position for 12 in the list.
Describe how you are going to assist him in this. Show the steps too. —
3

Total Marks:10**[CO3]**

Consider this graph:



- a. **Justify** $\sum_{v \in V} \deg(v) = 2m$, where m = number of edges in the graph. 2
- b. **Create** an adjacency list for this graph. 2
- c. **Apply** Breadth First Search to traverse this graph considering S to be the source vertex. **Demonstrate** the results. 6

Total marks:10

a. [CO7] **Calculate** the time complexity of the following function

Finding_Worst_Case(n):

```
int i,j,k,m,multi,a,b,c
for( i = n; i >= 1; i = i - 4 ){
    for( j = 1; j <= n; j = j * 3 ) {
        for( k=1; k<=20 ; k=k+1){
            multi=a*b
        }
        for( m=n ; m>=1 ; m=m/5 ){
            multi=multi*c
        }
    }
}
```

b.[CO7] **Calculate** the time complexity of the following recurrence relation.

[Any method is acceptable as long as steps are shown]

$$T(n) = T(n/3) + T(n/5) + n$$

BRAC UNIVERSITY
Department of Computer Science and Engineering

Examination: Mid Semester Exam
 Duration: 1 Hour 15 Minutes

Semester: Fall 2023
 Full Marks: 40

CSE 221: Algorithms

Answer the following questions.
 Figures in the right margin indicate marks.

Name:		ID:	Section:
1.	a. CO2	Explain the time complexity of the following code snippet in regards of the Big-O notation: <pre>1. for (i=0; i<n; i+=4) { 2. for (j=1; j<n; j*=2) { 3. for (k=0; k<30; k++) { 4. print("Am I still not 30?!!"); 5. } 6. print("Why, God, why? We had a Deal!"); 7. for (m=n; m>0; m-=2) { 8. print("Could you BE more dramatic?"); 9. } 10. } 11. }</pre>	04
	b. CO2	Consider the following functions. $f_1(n) = (\log n)^{2023}$ $f_2(n) = n^2 \log_n(n^n)$ $f_3(n) = n^3 + 7n^2$ $f_4(n) = 2.023^n$ $f_5(n) = n \log n$ $f_6(n) = n * \sqrt[3]{n^2}$ Now do the followings: a. Write a correct asymptotic upper bound for each of the above. b. Sort the functions in ascending order of their growth rate, assuming n is significantly large. Just write the sorted order, no need to show any simulation.	03 03

2.	CO3	Consider an array containing N unique values where for some index i, the values are in increasing order from index 0 to (i-1), and then again from i to (N-1). An example array is given below. Here i=3, it means the values are in increasing order from index 0 to 2, and then again from 3 to 7. <table><tr><td>index</td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td></tr><tr><td>value</td><td>9</td><td>12</td><td>15</td><td>2</td><td>4</td><td>5</td><td>7</td><td>8</td></tr></table> Given the value of i, propose an algorithm to search a <i>key_value</i> in the array. Complexity of your algorithm must be less than O(N). a) Present your solution idea as a code/ pseudocode/ flowchart/ step-by-step instructions/ logical explanations in short. b) Write the time complexity of your presented solution. c) Show a simulation of the Merge Sort algorithm to organize the whole array in increasing order.	index	0	1	2	3	4	5	6	7	value	9	12	15	2	4	5	7	8	04 01 05																															
index	0	1	2	3	4	5	6	7																																												
value	9	12	15	2	4	5	7	8																																												
3.	CO1	Your friend gave you a binary string B (meaning each character is either 0 or 1). He wanted to find out how to calculate the maximum number of consecutive 0s in that particular string. For example, <table><tr><td>String: 100100000111</td><td>Maximum consecutive 0s: 5</td></tr><tr><td>String: 1010101010101</td><td>Maximum consecutive 0s: 1</td></tr></table> You, as an algorithm enthusiast, know that this can be solved in linear time. However, your friend asked you to propose a Divide and Conquer approach a) Name a suitable Divide and Conquer algorithm for this task. b) Explain how you can apply that algorithm in this scenario. Present your idea in a pseudocode/programmable code/Flowchart/step-by-step instructions. c) Write the time complexity of your algorithm.	String: 100100000111	Maximum consecutive 0s: 5	String: 1010101010101	Maximum consecutive 0s: 1	02 06 02																																													
String: 100100000111	Maximum consecutive 0s: 5																																																			
String: 1010101010101	Maximum consecutive 0s: 1																																																			
4.	a. CO1	You have the following adjacency matrix for a graph. However, some of the entries are missing. Your job is to find these missing entries with the help of some clues. Then draw the graph. <table><tr><td></td><td><i>A</i></td><td><i>B</i></td><td><i>C</i></td><td><i>D</i></td><td><i>E</i></td><td><i>F</i></td></tr><tr><td><i>A</i></td><td>0</td><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td><i>B</i></td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td></tr><tr><td><i>C</i></td><td>0</td><td>1</td><td>0</td><td>1</td><td>0</td><td></td></tr><tr><td><i>D</i></td><td>0</td><td></td><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td><i>E</i></td><td>0</td><td>1</td><td>1</td><td>1</td><td>0</td><td>0</td></tr><tr><td><i>F</i></td><td>0</td><td>0</td><td></td><td>0</td><td></td><td>0</td></tr></table> Clues: - E can not be reached from B - D can be reached from A $2 \times E = 3 \times V$ (twice the no. of edges is equal to thrice the no. of vertices) - In-degree of B is not a prime number		<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>F</i>	<i>A</i>	0	1	0	0	0	0	<i>B</i>	0	0	0	0	0	1	<i>C</i>	0	1	0	1	0		<i>D</i>	0		0	0	0	0	<i>E</i>	0	1	1	1	0	0	<i>F</i>	0	0		0		0	06
	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>F</i>																																														
<i>A</i>	0	1	0	0	0	0																																														
<i>B</i>	0	0	0	0	0	1																																														
<i>C</i>	0	1	0	1	0																																															
<i>D</i>	0		0	0	0	0																																														
<i>E</i>	0	1	1	1	0	0																																														
<i>F</i>	0	0		0		0																																														
	b. CO2	Is an adjacency list more memory efficient than an adjacency matrix? Justify your reasoning with respect to directed vs undirected, weighted vs unweighted, sparse vs dense graphs.	04																																																	

BRAC UNIVERSITY
Department of Computer Science and Engineering

Examination: Mid Semester Exam
 Duration: 1 Hour 15 Minutes

Semester: Fall 2023
 Full Marks: 40

CSE 221: Algorithms

Answer the following questions.
 Figures in the right margin indicate marks.

Name:	ID:	Section:
-------	-----	----------

1. a. Explain the time complexity of the following code snippet in regards of the Big-O notation: **04**
CO2

```

1. for (i=0; i<n; i+=4) {
2.     for (j=1; j<n; j*=2) {
3.         for (k=0; k<20; k++) {
4.             print("Am I still not 30?!!");
5.         }
6.         print("Why, God, why? We had a Deal!");
7.         for (m=n; m>0; m-=4) {
8.             print("Could you BE more dramatic?");
9.         }
10.    }
11. }
```

- b. Consider the following functions.
CO2

$$f_1(n) = (\log n)^{2000}$$

$$f_2(n) = n^3 \log_n(n^n)$$

$$f_3(n) = n^3 + 7n^2$$

$$f_4(n) = 4^n$$

$$f_5(n) = n \log n$$

$$f_6(n) = n * \sqrt{n}$$

Now do the followings:

- Write a correct asymptotic upper bound for each of the above. **03**
- Sort the functions in ascending order of their growth rate, assuming n is significantly large. **03**
 Just write the sorted order, no need to show any simulation.

2. **CO3** Consider an array containing **N** unique values where for some index **i**, the values are in increasing order from index **0** to (**i-1**), and then again from **i** to (**N-1**). An example array is given below. Here **i=4**, it means the values are in increasing order from index 0 to 3, and then again from 4 to 7.

index	0	1	2	3	4	5	6	7
value	9	12	15	20	4	5	7	8

Given the value of **i**, propose an algorithm to search a *key_value* in the array. Complexity of your algorithm must be less than **O(N)**.

- Present your solution idea as a code/ pseudocode/ flowchart/ step-by-step instructions/ logical explanations in short. **04**
- Write the time complexity of your presented solution. **01**
- Show a simulation of the Merge Sort algorithm to organize the whole array in increasing order. **05**

3. **CO1** Your friend gave you a binary string B (meaning each character is either **0** or **1**). He wanted to find out how to calculate the maximum number of consecutive 0s in that particular string. For example,

String: 1001000000111	Maximum consecutive 0s: 6
String: 10101010100101	Maximum consecutive 0s: 2

You, as an algorithm enthusiast, know that this can be solved in linear time. However, your friend asked you to propose a Divide and Conquer approach

- Name a suitable Divide and Conquer algorithm for this task. **02**
- Explain how you can apply that algorithm in this scenario. Present your idea in a pseudocode/programmable code/Flowchart/step-by-step instructions. **06**
- Write the time complexity of your algorithm. **02**

4. **a.** You have the following adjacency matrix for a graph. However, some of the entries are missing. **06**
CO1 Your job is to find these missing entries with the help of some clues. Then draw the graph.

	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>F</i>	
<i>A</i>	0	0		0	0	0	
<i>B</i>	1	0	1	0	0	1	
<i>C</i>	0	0	0	0	1	0	
<i>D</i>	0	0	1	0	0	0	
<i>E</i>	0		0	0	0		
<i>F</i>	1	0	1	0		0	

Clues:

- B can not be reached from C
- A can be reached from D
- $2 \times |E| = 3 \times |V|$
(twice the no. of edges is equal to thrice the no. of vertices)
- In-degree of C is not a prime number.

- b.** Is an adjacency list more memory efficient than an adjacency matrix? **04**
CO2 Justify your reasoning with respect to directed vs undirected, weighted vs unweighted, sparse vs dense graphs.

BRAC UNIVERSITY
Department of Computer Science and Engineering

Examination: Mid Semester Exam
 Duration: 1 Hour 15 Minutes

Semester: Fall 2024
 Full Marks: 30

CSE 221: Algorithms

Answer the following questions.
 Figures in the right margin indicate marks.

Name:	ID:	Section:
-------	-----	----------

- 1 a. Consider the following code snippet to answer questions (i) and (ii).
CO2

```

1. Function F(n):
2.     If (n == 1):
3.         Return True;
4.     Else:
5.         X = F(n/2);
6.         Y = F(n/2);
7.         Z = F(n/2);
8.         U = A(n);
9.
10. Function A(n):
11.     For i := n to 1; i--:
12.         Return True;
```

- i. **Write** the time complexity of the function “F” using a recurrence relation. **02**
 ii. **Express** the recurrence relation from question (i) with asymptotic upper bound (Big-O) using any convenient method. Show your work. **02**

- b. Given an array of **N** elements, you are required to find the **peak element** in it. An element is considered to be a peak if its value is greater than the value(s) of its adjacent element(s) i.e. immediate left and immediate right. If there is only one adjacent element, we take only that in consideration. And the array will contain **exactly one** peak element.

Now answer the following questions.

- CO1** i. **Determine** whether each of the following arrays matches the above stem-mentioned conditions: **02**
 1. [1, 5, 4, 3, 2]
 2. [1, 3, 2, 5, 4]
 3. [5, 2, 1, 3, 4]
 4. [1, 2, 3, 4, 5]
 [N.B.: For each array, it is enough to write only True/False in the answer script.]
- CO3** ii. **Present** an algorithm with a time complexity lower than **O(N)** to solve the stem-mentioned problem using pseudocode/ programmable code/ step-by-step logical instructions. **03**
- CO2** iii. If the array contains **more than one** peak element, will the presented algorithm in question (ii) still be able to find any peak element? **Explain** briefly. **01**

- 2 a. You are designing a computerized logistics system for a warehouse that handles thousands of packages daily. Each package is assigned a unique package ID and a delivery deadline (in terms of 'remaining hours' from the current time) and the packages are initially sorted according to their package_id. The warehouse management system needs to sort the packages by their delivery deadlines in ascending order to prioritize the earliest deliveries. 05
- CO1 But there are some conditions to develop this system. The system does not have any issues related to space/memory but it needs the list to be sorted with time complexity $O(n \log n)$ where n is the number of packages to process. Moreover, if any two packages have the same delivery deadline, the package with smaller package_id comes before i.e the algorithm must be stable.
- Write the name of the appropriate algorithm for this case and show the solution with pseudocode/programmable code/step-by-step logical instructions.
- b. [Continues on the previous question...] After some days, the warehouse noticed a massive increase in daily orders and the system was facing memory issues to hold these massive orders. So they consulted you again. You suggested to them an algorithm which does not require any additional memory. But you also warned them that it can degrade to the time complexity of $O(n^2)$ in some cases. However, they insist the time complexity must remain $O(n \log n)$ in all cases. So you proposed a modification to your algorithm for question (a) without increasing the time complexity.
- CO1 i. Name the algorithm you suggested them and explain the case where the algorithm might take $O(n^2)$ time. 03
- CO2 ii. What solution did you come up with to ensure $O(n \log n)$? Explain where it differs from the solution of question (a). 02
- 3 A football team's analytics department hired you and gave you a task of reviewing match performance data from the past season. Each match's net goal difference (goals scored minus goals conceded) is recorded in a list. Now you were asked by the department to identify the team's best consecutive performance streak, where the net goal difference was maximized over a series of matches.
- net_goal_difference: [1, 4, 3, -5, 5, 6, 1, -4]
- To prove your skills, you used an approach where the problem is divided into subproblems to find the solution. Now tell us more about it by answering the following questions.
- CO1 i. For the given input above, determine the number of times the *Cross-Sum* exceeds both the *Left-Sum* and the *Right-Sum*. Exclude base cases (subarrays of size 1) from consideration. Show your work properly. 06
Cross-Sum: sum of elements calculated by combining sum from mid to left and mid to right of the current subarray.
Left-Sum: maximum sum in the left half of the subarray.
Right-Sum: maximum sum in the right half of the subarray.
- CO3 ii. The department wants you to find the maximum consecutive matches with a positive goal difference. Now present your idea to solve the problem considering you have to use the same approach used earlier with pseudocode/programmable code/step-by-step logical instructions. 04

BRAC UNIVERSITY
Department of Computer Science and Engineering

Examination: Mid Semester Exam
 Duration: 1 Hour 15 Minutes

Semester: Fall 2024
 Full Marks: 30

CSE 221: Algorithms

Answer the following questions.
 Figures in the right margin indicate marks.

Name:	ID:	Section:
-------	-----	----------

- 1 a. Consider the following code snippet to answer questions (i) and (ii).
CO2

```

1. Function F(n):
2.   If (n == 1):
3.     Return True;
4.   Else:
5.     U = F(n/3);
6.     X = F(n/3);
7.     Y = F(n/3);
8.     Z = F(n/3);
9.     V = A(n);
10.
11. Function A(n):
12.   Sum = 0
13.   For i := 1 to n; i++:
14.     Sum = Sum + i;
15.   Return sum;
```

- i. **Write** the time complexity of the function “F” using a recurrence relation. **02**
 ii. **Express** the recurrence relation from question (i) with asymptotic upper bound (Big-O) using any convenient method. Show your work. **02**

- b. Given an array of N elements, you are required to find the **valley element** in it. An element is considered to be a valley if its value is less than the value(s) of its adjacent element(s) i.e. immediate left and immediate right. If there is only one adjacent element, we take only that in consideration. It is guaranteed that the array contains only **one** valley element.

Now answer the following questions.

- CO1** i. **Determine** whether each of the following arrays matches the above stem-mentioned conditions: **02**
 1. [5, 3, 2, 1, 4]
 2. [3, 1, 5, 2, 4]
 3. [1, 2, 3, 4, 5]
 4. [1, 3, 5, 4, 2]
 [N.B.: For each array, it is enough to write only True/False in the answer script.]
- CO3** ii. **Present** an algorithm with a time complexity lower than **O(N)** to solve the stem-mentioned problem using pseudocode/ programmable code/ step-by-step logical instructions. **03**
- CO2** iii. If the array contains **more than one** peak element, will the presented algorithm in question (ii) still be able to find any peak element? **Explain** briefly. **01**

- 2 a. You are designing a computerized logistics system for a warehouse that handles thousands of packages daily. Each package is assigned a unique package ID and a delivery deadline (in terms of 'remaining hours' from the current time) and the packages are initially sorted according to their package_id. The warehouse management system needs to sort the packages by their delivery deadlines in ascending order to prioritize the earliest deliveries. **05**
- CO1**

But there are some conditions to develop this system. The system does not have any issues related to space/memory but it needs the list to be sorted with time complexity $O(n \log n)$ where n is the number of packages to process. Moreover, if any two packages have the same delivery deadline, the package with smaller package_id comes before i.e the algorithm must be stable.

Write the name of the appropriate algorithm for this case and show the solution with pseudocode/programmable code/step-by-step logical instructions.

- b. [Continues on the previous question...] After some days, the warehouse noticed a massive increase in daily orders and the system was facing memory issues to hold these massive orders. So they consulted you again. You suggested to them an algorithm which does not require any additional memory. But you also warned them that it can degrade to the time complexity of $O(n^2)$ in some cases. However, they insist the time complexity must remain $O(n \log n)$ in all cases. So you proposed a modification to your algorithm for question (a) without increasing the time complexity.

- CO1** i. **Name** the algorithm you suggested them and explain the case where the algorithm might take $O(n^2)$ time. **03**
- CO2** ii. What solution did you come up with to ensure $O(n \log n)$? **Explain** where it differs from the solution of question (a). **02**

- 3 A stock trading firm hired you to analyze daily stock price changes of a company over the past quarter. Each day's net price change (price at the end of the day minus price at the start) is recorded in a list. You were asked by the firm to identify the company's best consecutive stock price increase, where the price change was maximized over a series of days.

Price_changes = [2, 3, 5, -7, 3, 2, 1, -6]

To prove your skills, you used an approach where the problem is divided into subproblems to find the solution. Now tell us more about it by answering the following questions.

- CO1** i. For the given input above, **determine** the number of times the *Cross-Sum* exceeds both the *Left-Sum* and the *Right-Sum*. Exclude base cases (subarrays of size 1) from consideration. Show your work properly. **06**
Cross-Sum: sum of elements calculated by combining sum from mid to left and mid to right of the current subarray.
Left-Sum: maximum sum in the left half of the subarray.
Right-Sum: maximum sum in the right half of the subarray.
- CO3** ii. The firm wants you to find the maximum consecutive days with a positive price change. Now **present** your idea to solve the problem considering you have to use the same approach used earlier with pseudocode/programmable code/step-by-step logical instructions. **04**

BRAC UNIVERSITY
Department of Computer Science and Engineering

Examination: Midterm Exam
 Duration: 1 Hour 20 minutes

Semester: Fall 2025
 Full Marks: 25

CSE 221: Algorithms

Answer the following questions.
 Figures in the right margin indicate marks.

Name:	ID:	Section:
-------	-----	----------

1 a. Why do we ignore constants and lower-order terms when expressing the time complexity of an algorithm? Briefly **explain** in your own words. **01**
 CO2

b. Is $n * \sqrt{n}$ asymptotically faster than $n * \log_2(n)$? **Justify** with proper logic. **02**
 CO2

c. **Write** the time complexity of the following code snippet. **03**
 CO2

```
public static void func(int[] arr) {
    int n = arr.length;
    int k = 0;
    for (int i = 0; i < n; i++)
        k += arr[i];

    for (int i = 1; i < n; i++)
        for (int j = 1; j <= arr[i]; j++)
            System.out.print(i);
}
```

2 a. You are given an array A consisting of N integers. Some of the numbers are positive, and some are negative. For example A : {15, - 13, 3, - 4, 5, - 6}. **05**
 CO1

You have to organize the array in a way that all the negative numbers come to the front and all positive numbers are pushed to the back. For example, the array A from above becomes *Org_A* : {- 6, - 4, - 13, 15, 3, 5}. *Note that numbers don't need to be sorted here.*

Propose a linear-time [$O(N)$] algorithm with constant [$O(1)$] space complexity. **Present** your algorithm as pseudocode or executable code, or step-by-step logical instructions.

If unable, show a simulation of any algorithm (no limit on the time or space complexity) on the given example array A for 40% marks.

b. After solving Q2(a), you want to find the index of the leftmost positive number in the organized array *Org_A*. Present a modification of Binary Search to achieve this. **Write** either pseudocode or executable code. **03**
 CO1

- c. CO2 Mention the name and time complexity of the most efficient sorting algorithm among Bubble, Insertion, Selection, Merge, Quick, and Count for each of the following input scenarios, consisting of an array of N integers. For each case, you do not have any information other than the given one.

Case #1: The largest number can be in any index, but all other numbers are already sorted.

Case #2: The smallest number is 0, and the largest is 2^N .

Case #3: The array contains all the integers from 1 to N .

Case #4: The numbers are in the reverse order of how we want them to be sorted.

- 3 We are given an array A consisting of N integers. Some of the numbers are positive, and some are negative. For example $A : \{15, -13, 3, -4, 5, -6\}$.

We want to find the **Max-Prod Subarray** of the input. Just like the Max-sum Subarray, but instead of summation, we have to consider the product (multiplication) of the numbers.

We have come up with a divide-and-conquer algorithm to solve this problem. This algorithm has the same concept that you know for the Max-sum Subarray, just modified for multiplication.

```
function MaxProdSubarray(A, left, right){
    if (left == right):
        return A[left]

    mid = (left + right) / 2

    LPS = MaxProdSubarray(A, left, mid)
    RPS = MaxProdSubarray(A, mid + 1, right)
    CPS = CrossProduct(A, left, mid, right)

    return max(LPS, RPS, CPS)
}

function CrossProduct(A, left, mid, right){
    leftProd = -∞
    prod = 1
    for (i from mid to left){
        prod = prod * A[i]
        if (prod > leftProd): leftProd = prod
    }

    rightProd = -∞
    prod = 1
    for (i from mid+1 to right){
        prod = prod * A[i]
        if (prod > rightProd): rightProd = prod
    }
    return leftProd * rightProd
}
```

- a. **Mark** the changes that you see in this algorithm than the one for Max-Sum Subarray. **02**
CO1 You can either write only those lines in your script. Or, write the whole code given above and underline the ones you want to mark. Also, you don't have to mark the changes in variable and function names.
- b. **Evaluate** whether the given algorithm would actually work for this problem. **03**
CO3 If your answer is yes, give a proper logical explanation. Otherwise, give an input where the algorithm will not produce the correct answer.
- c. We are thinking, why not use Karatsuba's multiplication technique to make our algorithm **02**
CO2 faster? So we replace the multiplication operations (*) with Karatsuba's function. For example,
 $prod * A[i]$ becomes $Karatsuba(prod, A[i])$.
- Write** a recurrence relation [$T(N) = ___ + ___$] to express the time complexity of the algorithm after applying Karatsuba. Assume that Karatsuba's time complexity is $O(N^{1.58})$ (same N as the size of the Array A).
- d. Now **solve** your recurrence from Q3(c) to find the actual time complexity. **02**
CO2 Any method is acceptable as long as you show proper work.

BRAC UNIVERSITY
Department of Computer Science and Engineering

Examination: Midterm Exam
 Duration: 1 Hour 20 minutes

Semester: Fall 2025
 Full Marks: 25

CSE 221: Algorithms

Answer the following questions.
 Figures in the right margin indicate marks.

Name:	ID:	Section:
-------	-----	----------

1 a. Why do we ignore constants and lower-order terms when expressing the time complexity of an algorithm? Briefly **explain** in your own words. 01
 CO2

b. Is $\sqrt{n} * \log_2(n)$ asymptotically faster than n ? **Justify** with proper logic. 02
 CO2

c. **Write** the time complexity of the following code snippet. 03
 CO2

```
public static void func(int[] arr) {
    int n = arr.length;
    int k = 0;
    for (int i = 0; i < n; i++)
        k += arr[i];

    for (int i = 1; i < n; i++)
        for (int j = 1; j <= k; j++)
            System.out.print(i);
}
```

2 a. You are given an array B consisting of N integers. Some of the numbers are even, and some are odd. For example $B : \{7, 2, 9, 4, 11, 6\}$. 05
 CO1

You have to organize the array in a way that all the even numbers come to the front and all odd numbers are pushed to the back. For example, the array B from above becomes $Org_B : \{6, 2, 4, 9, 11, 7\}$. *Note that numbers don't need to be sorted here.*

Propose a linear-time $[O(N)]$ algorithm with constant $[O(1)]$ space complexity. **Present** your algorithm as pseudocode or executable code, or step-by-step logical instructions.

If unable, show a simulation of any algorithm (no limit on the time or space complexity) on the given example array B for 40% marks.

b. After solving Q2(a), you want to find the index of the rightmost even number in the organized array Org_B . Present a modification of Binary Search to achieve this. **Write** either pseudocode or executable code. 03
 CO1

- c. Mention the name and time complexity of the most efficient sorting algorithm among Bubble, Insertion, Selection, Merge, Quick, and Count for each of the following input scenarios, consisting of an array of N integers. For each case, you do not have any information other than the given one.

CO2

Case #1: The smallest number can be in any index, but all others are already sorted.

Case #2: The smallest number is 0, and the largest is 2^N .

Case #3: The array contains all the integers from 1 to N .

Case #4: Just some random numbers, no pattern could be seen.

3

We are given an array A consisting of N integers. Some of the numbers are positive, and some are negative. For example $A : \{5, -3, 13, 4, -5, -6\}$.

We want to find the **Max-Prod Subarray** of the input. Just like the Max-sum Subarray, but instead of summation, we have to consider the product (multiplication) of the numbers.

We have come up with a divide-and-conquer algorithm to solve this problem. This algorithm has the same concept that you know for the Max-sum Subarray, just modified for multiplication.

```
function MaxProdSubarray(A, left, right){
    if (left == right):
        return A[left]

    mid = (left + right) / 2

    LPS = MaxProdSubarray(A, left, mid)
    RPS = MaxProdSubarray(A, mid + 1, right)
    CPS = CrossProduct(A, left, mid, right)

    return max(LPS, RPS, CPS)
}

function CrossProduct(A, left, mid, right){
    leftProd = -∞
    prod = 1
    for (i from mid to left){
        prod = prod * A[i]
        if (prod > leftProd): leftProd = prod
    }

    rightProd = -∞
    prod = 1
    for (i from mid+1 to right){
        prod = prod * A[i]
        if (prod > rightProd): rightProd = prod
    }
    return leftProd * rightProd
}
```

- a. **Mark** the changes that you see in this algorithm than the one for Max-Sum Subarray. **02**
CO1 You can either write only those lines in your script. Or, write the whole code given above and underline the ones you want to mark. Also, you don't have to mark the changes in variable and function names.
- b. **Evaluate** whether the given algorithm would actually work for this problem. **03**
CO3 If your answer is yes, give a proper logical explanation. Otherwise, give an input where the algorithm will not produce the correct answer.
- c. We are thinking, why not use Karatsuba's multiplication technique to make our algorithm **02**
CO2 faster? So we replace the multiplication operations (*) with Karatsuba's function. For example,
 $prod * A[i]$ becomes $Karatsuba(prod, A[i])$.
Write a recurrence relation [$T(N) = __ + __$] to express the time complexity of the algorithm after applying Karatsuba. Assume that Karatsuba's time complexity is $O(N^{1.58})$ (same N as the size of the Array A).
- d. Now **solve** your recurrence from Q3(c) to find the actual time complexity. **02**
CO2 Any method is acceptable as long as you show proper work.